

# Pause and Resume on blockwise transfers for LwM2M

Authors: Jaime Jiménez, Emil Zhang, Daniel Zhang

## OMA DMSO, Ericsson Proposal

|          |                                                   |          |
|----------|---------------------------------------------------|----------|
| <b>1</b> | <b>TECHNICAL BACKGROUND .....</b>                 | <b>2</b> |
| 1.1      | REQUIREMENTS DESCRIPTION .....                    | 3        |
| <b>2</b> | <b>SOLUTION DESCRIPTION.....</b>                  | <b>4</b> |
| 2.1      | SAVING THE STATE OF A TRANSFER IN LWM2M.....      | 4        |
| 2.2      | PAUSING THE STATE OF A TRANSFER IN LWM2M .....    | 5        |
| 2.3      | RECOVERING THE STATE OF A TRANSFER IN LWM2M ..... | 6        |
| 2.4      | LWM2M MANAGER-DRIVEN FOTA RESUME.....             | 6        |
| 2.5      | ADAPTING TO HIGH BIT ERROR RATE.....              | 7        |
| 2.6      | TRANSFERRED BLOCKS .....                          | 8        |

# 1 Technical Background

The Constrained Application Protocol (CoAP) is a generic REST application Protocol for constrained devices. CoAP is designed to be used over UDP (RFC768) over the Internet. CoAP uses a short fixed-length binary header (4 bytes) (see Figure 1).

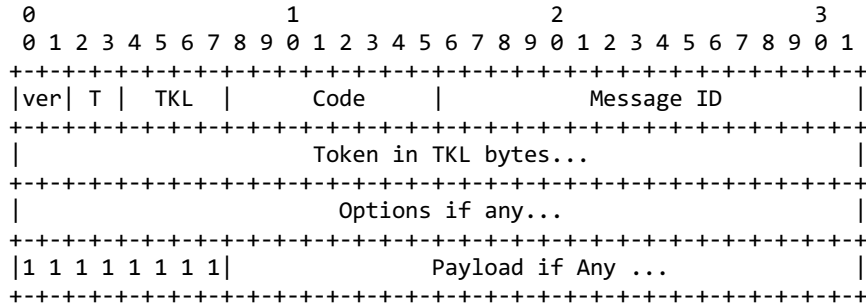


Figure 1: CoAP Header

CoAP was originally intended for UDP transmission, which is unreliable. This means that CoAP request and response messages may arrive out of order, appear duplicated, or go missing without notice. For this reason, CoAP implements a lightweight reliability mechanism using four different message types, including “confirmable” and “non-confirmable” messages.

To allow for larger payload transfer over UDP, Block-Wise Transfers in CoAP where created. In contrast to HTTP, where TCP does the grunt work of segmenting and resequencing, CoAP is based on datagram transports such as UDP or Datagram Transport Layer Security (DTLS). These transports only offer fragmentation, which is even more problematic in constrained nodes and networks, limiting the maximum size of resource representations that can practically be transferred.

Instead of relying on IP fragmentation, blockwise transfer extends basic CoAP with a pair of “Block” options for transferring multiple blocks of information from a resource representation in multiple request-response pairs.

Three items of information may need to be transferred in a Block (Block1 or Block2) option: the size of the block (SZX), whether more blocks are following (M) and the relative number of the block (NUM) within a sequence of blocks with the given size. The minimum block size is 16 and the maximum is 1024 bytes.

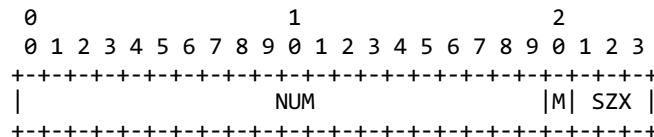
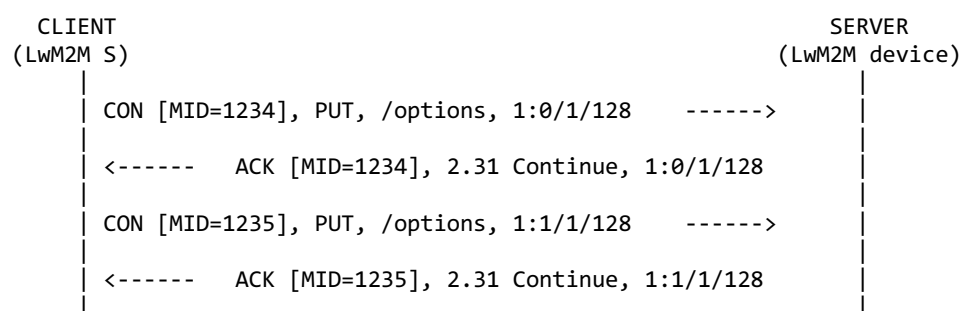


Figure 2: Block Option Value

The size of the block: 16, 32, 64, 128, 256, 512 and 1024 and SZX is a way to represent it as a three-bit unsigned integer indicating the size of a block to the power of two.

Blockwise defines new response status codes: 2.31 Continue when the transfer was continued, and the server encourages sending further blocks. 4.08 Request Entity Incomplete client error status code indicates that the server has not received the blocks of the request body that it needs to proceed.

Blockwise can be used with CoAP's GET, POST, PUT methods and allows for negotiating the block size. Below is an example from [RFC7959](#):



*Figure 3: Simple Atomic Block-Wise PUT*

In the example a client does a PUT operation on a CoAP Server. The expression 1:0/1/128 indicates the Block option 1 the current block number 0, whether more blocks are coming (*M bit*) 1 and the size of the block 128.

In the acknowledgment the server is supposed to indicate the block it has received and to tell the client to continue the transfer with 2.31. When concluded it must reply with the normal 2.04.

The Message ID (MID) field is used to detect message duplication and to match messages of type Acknowledgement/Reset to messages of type Confirmable/Non-confirmable

Finally, [RFC7959](#) also defines a CoAP option to tell in advance the total size of the file being transferred (size1 for requests and size2 for responses).

## 1.1 Requirements Description

Over LwM2M, when a firmware update is initiated, the firmware image is split in chunks with a predefined size. The created blocks are sent to the LwM2M endpoint, which identifies them by Message ID. The size of the block is defined by the LwM2M server (e.g., 128 bytes) and the image size may range from as little as 80 KiB to few MiB in size. For example, if a firmware image has a total size of 80 KiB and is split in chunks of 128 bytes, then the LwM2M server needs to send 640 messages to the LwM2M endpoint. Once the firmware image has been transferred, the LwM2M-based firmware update is initiated.

It has been found complicated to implement blockwise transfer in LwM2M. The reason being that the LwM2M specification does not allow for pause and resume

of existing blockwise transfers. Since firmware updates may take from few minutes to an hour to happen, it often occurs that the connection breaks in between. During these episodes, the CoAP Client can lose the state of the transfer making it convenient to be able to resume the transfer once the connection recovers.

Over LwM2M, when a firmware update has been initiated, the LwM2M server may need to support discontinuous FOTA/SOTA data transfer in scenarios when the LwM2M Server (CoAP Client) loses the state of the transfer, often after a protracted pause, which may happen:

- When the LwM2M server detects that the battery level of a device cannot afford the remaining data transfer, it wants to pause the on-going data transfer session until the device is re-charged.
- When a device detects that current signal strength is too weak to keep a stable, continuous data transfer, we would like to pause the transfer or signal the LwM2M server to pause the on-going data transfer session until the signal strength is back to normal.

As a general rule, for all the cases mentioned above:

- Firstly, it will be good to keep the retransmitted data as little as possible to reduce the network load as well as the power consumption of the devices. This requirement is also known as *"Breakpoint Continuity"*.
- Secondly, if one party wants to pause the data transfer session, it will be good to have a mechanism to tell the other party explicitly over LwM2M in this case- so that the later does not need to wait MAX\_TRANSMIT\_WAIT (RFC7252, default value is 93 seconds). Especially for devices, waiting means significant power consumption.

## 2 Solution Description

### 2.1 Saving the state of a transfer in LwM2M

A new object called FOTA\_STATE would contain the information of a FOTA transfer. The object may be used for other types of large file transfers.

```
OBJECT: FOTA_STATE
+--ro FOTA_STATE*
+--rw instance number

+--rw FotaID
+--rw FotaSize (in KB)
+--rw FotaPath
+--rw BlockOpt
+--rw BlockSize
+--rw LastBlock
+--rw BlockArray
```

Figure 4: LwM2M Object to indicate the FOTA state

The object contains an identifier of the specific FOTA transfer if the instance number is not sufficient, the expected total size of the FOTA, the path where the file is being stored, the Block Option being used, the Block Size and the Last Block that has successfully been transferred. The last block is always tied to a specific block size, this is because the resource representation that is partitioned will vary depending on the block (e.g., 80KB can be 80 blocks of size 1024 or 320 blocks of size 256).

## 2.2 Pausing the state of a transfer in LwM2M

Continuing with the example on Figure 3, we have a transfer from a CoAP Client (LWM2M Manager) to a CoAP Server (LwM2M Client), and a Firmware of 80 KB with a block size of 1024.

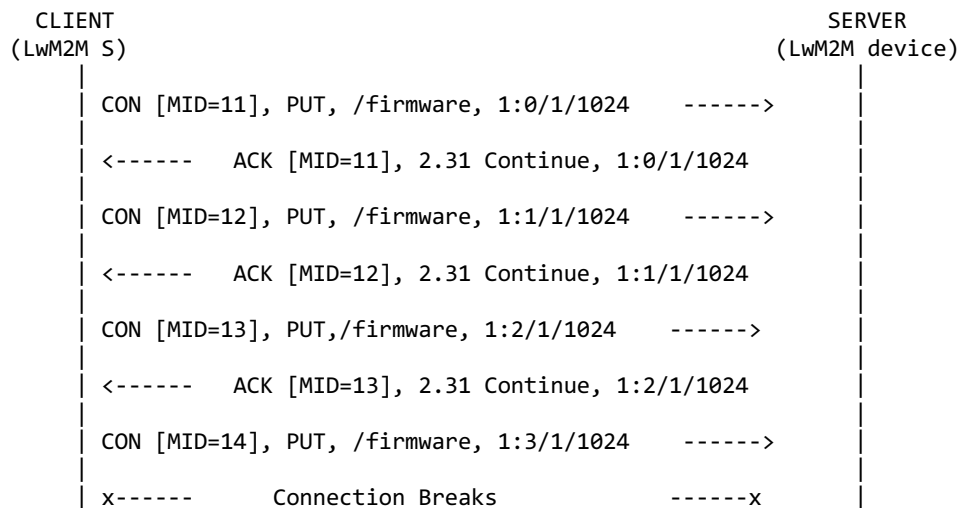
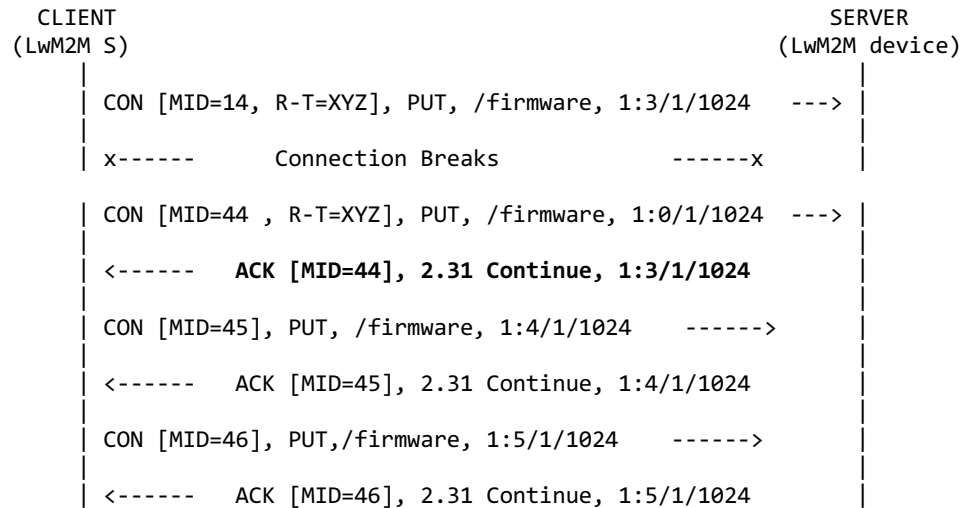


Figure 5: FOTA of 80KB with 1024 block size breaks

The connection breaks after 3 blocks are transferred and the last CON message is never replied. At that point the CoAP Server has 3 blocks of 1024 bytes each and the Client knows that. Over LwM2M a new resource [PAUSE](#) in the [LwM2M FOTA Object](#) could trigger this interruption on the client.

## 2.3 Recovering the state of a transfer in LwM2M

To resume the service after reconnecting we envision a handshake in which the client tries to start normally but the Server replies with the last block received, that is the last ACK with a MID equal to the current request as shown in the figure below.



*Figure: FOTA of 80KB with 1024 block size resumes using the same request tag*

In this case the CoAP server replies with 2.31 Continue, 1:3/1/1024 instead of 2.31 Continue, 1:0/1/1024, the expectation is for the CoAP Client to adjust accordingly and continue from the block the server is signaling.

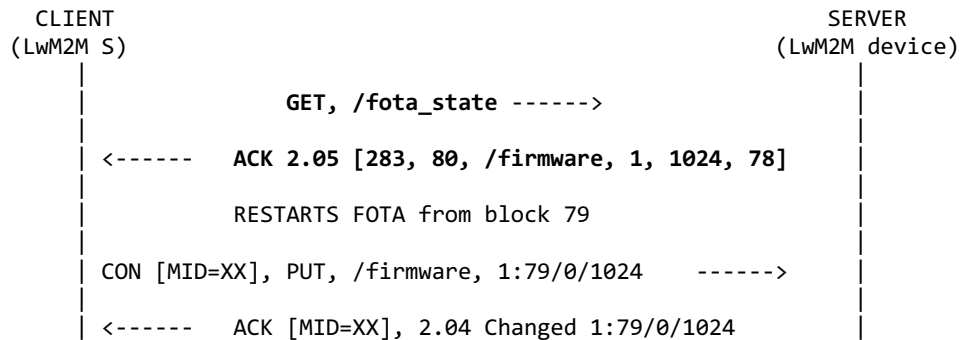
For the server behavior to happen as described the same request-tag needs to be used. The expectation is for the CoAP Client to adjust accordingly and continue from the block the server is signaling. In the figure the tag has the value "XYZ" to identify the fragmented message, aiding in the correlation of message blocks and ensuring that the server knows that the first request refers to the previous incompletely transferred body and not to a new transfer on the same resource path.

Over LwM2M a new resource `RESUME` in the LwM2M FOTA Object (/5) can trigger this continuation. The LwM2M Client MUST keep the information about the last block and the block size. For this reason, it is mandatory that in LwM2M the `FOTA_STATE` object exists and that it can signal the right current block to the client.

## 2.4 LwM2M Manager-driven FOTA resume

Optionally, the LwM2M client (CoAP Server) might be too simple to be able to adapt to different FOTA blockwise modes. In those cases, the LwM2M Manager (CoAP Client) will be the one driving the whole FOTA process by means of the `FOTA_STATE` object.

The CoAP Server will be exposing the FOTA\_STATE as an observable resource and a manager may at any point read it, for example in case the state of the FOTA failed.



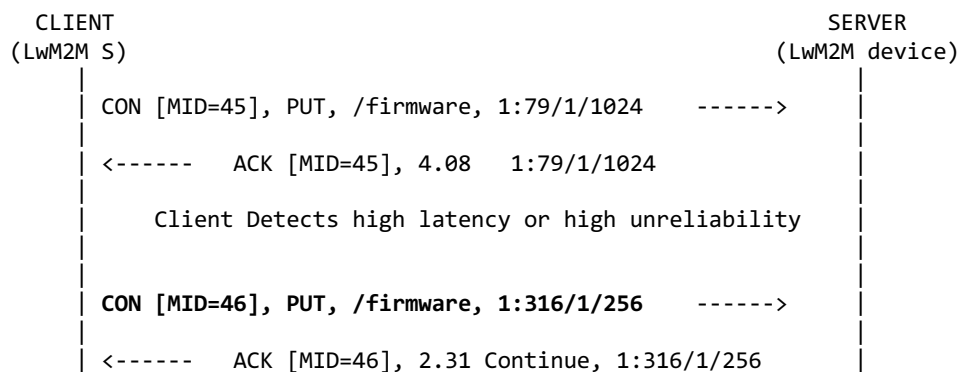
*Figure 8: Client request current FOTA state and resumes FOTA.*

In this scenario, the CoAP Client (LwM2M Manager) requests from the CoAP server (LwM2M Client or IoT device) its current fota state. The client replies with the information (i.e., stored FotaID is 283, 80KB of firmware size, /firmware for the path of the firmware, Blockwise Option 1 cause PUT was the last method used, block size of 1024 and last successful block being block 78).

At that point the Manager may simply continue with the same parameters to conclude the FOTA, by sending the last block (79).

## 2.5 Adapting to high bit error rate

During transfer, due to poor network reliability the error rate may go up. In those cases, it is important to be able to signal that a different block size is required. A way to do this is to use the FotaSize to be able to adjust the transfer rate to ensure a reliable transfer.



*Figure 7: FOTA of 80KB is almost complete but fails on the last block. Client switches to a smaller block size of 256.*

In the example, the FOTA process starts failing (e.g., by returning 4.08 error codes) and the client adapts the transfer to have lower block sizes which have a higher chance of success.

Over LwM2M this may be implemented by the same PAUSE/RESUME mechanism and by a modification of the block size, providing that the client has exchanged the total file size with the server over the CoAP Option size1. This way the server can recalculate the new blocksize and append the smaller block to the last block sent that did not fail (i.e., block 78 with size 1024).

## 2.6 Transferred blocks

To solve the issue of signaling gaps in the transfer, we will update the FOTA\_STATE object with the BlockArray resource.

The BlockArray contains per-block state about the blockwise transfer status. If a block has been successfully received the bit is set to "1", if it is not it is set to "0".

As an example, for a transferred of 10KB with a block size of 1024 bytes, the initial empty BlockArray would contain the following: [0,0,0,0,0,0,0,0,0,0]. Once completed it will be [1,1,1,1,1,1,1,1,1,1]. If there were 2 missing blocks (3 and 5), the observed array would be [1,1,0,1,0,1,1,1,1,1].

### Observing BlockArray

In CoAP, you can use query parameters to specify conditions for your Observe request. We can thus observe the state of this resource with an expected final state, (e.g., "state=1111111111") that the client wishes to be notified of. Therefore when the bit array state becomes [1,1,1,1,1,1,1,1,1,1] the FOTA transfer will be complete and the client will receive a notification.